# BANCO DE DADOS

com Microsoft SQL Server

## AULA 09

## Ordenação e Agrupamento

ORDER BY · GROUP BY · HAVING

ORDER BY | GROUP BY | HAVING | WHERE vs HAVING | ROLLUP | Relatórios | Métricas | Window Fn

# 🎯 Objetivos da Aula 09

- ⬆️ Dominar ORDER BY: múltiplas colunas, expressões e NULLs
- 📋 Entender a ordem completa de execução do SELECT
- 📊 Agrupar dados com GROUP BY para relatórios gerenciais
- 📈 Criar relatórios de faturamento, ticket médio e rankings
- 🔢 Aplicar funções de agregação com grupos (COUNT, SUM, AVG...)
- 🗂️ Gerar subtotais automáticos com ROLLUP
- 🔍 Filtrar grupos com HAVING (≠ WHERE que filtra linhas)
- 💡 Introdução a Window Functions: ROW_NUMBER e RANK

# 🔄 Revisão — Aula 08: UPDATE, DELETE e Transações

❓ **Por que sempre usar WHERE no UPDATE e DELETE?**

✅ Sem WHERE, o comando afeta TODOS os registros da tabela — pode destruir os dados

❓ **Qual a diferença entre TRUNCATE e DELETE?**

✅ TRUNCATE remove tudo, reseta IDENTITY, sem log/triggers. DELETE é mais controlado

❓ **O que é ACID?**

✅ Atomicidade, Consistência, Isolamento e Durabilidade — propriedades de transações

❓ **O que faz ROLLBACK?**

✅ Desfaz todas as operações desde o BEGIN TRANSACTION — como Ctrl+Z do banco

❓ **Para que serve TRY/CATCH em SQL?**

✅ Captura erros automaticamente e executa o CATCH (normalmente com ROLLBACK)

# Ordem de Execução — O SQL por Dentro

A ordem de EXECUÇÃO é diferente da ordem de ESCRITA. Entender isso explica por que certas coisas funcionam e outras não — como o HAVING após GROUP BY e o alias no WHERE.

1

**FROM**

Identifica a(s) tabela(s) — JOINs são resolvidos aqui

2

**WHERE**

Filtra LINHAS antes do agrupamento — não usa funções de agregação

3

**GROUP BY**

Agrupa as linhas filtradas em grupos pelo(s) valor(es) informado(s)

4

**HAVING**

Filtra GRUPOS — pode usar funções de agregação (COUNT, SUM...)

5

**SELECT**

Seleciona/calcula as colunas — alias são definidos aqui

6

**DISTINCT**

Elimina duplicatas do resultado do SELECT

7

**ORDER BY**

Ordena o resultado — pode usar alias do SELECT

8

**OFFSET/FETCH**

Pagina o resultado ordenado

# 💻 ORDER BY — Ordenação Avançada

```sql
-- Ordenação simples: ASC (padrão) e DESC
SELECT nome_produto, preco FROM PRODUTOS
ORDER  BY preco DESC;  -- mais caro primeiro

-- Múltiplas colunas de ordenação (prioridade da esquerda)
SELECT nome, estado, cidade FROM CLIENTES
ORDER  BY estado ASC, cidade ASC, nome ASC;

-- Ordenar por expressão calculada
SELECT nome_produto, preco, estoque,
       preco * estoque AS valor_estoque
FROM   PRODUTOS
ORDER  BY preco * estoque DESC;  -- ou pelo alias: valor_estoque DESC

-- NULLs: por padrão vão ao FINAL no ASC e ao INÍCIO no DESC
SELECT nome, email FROM CLIENTES ORDER BY email ASC;
-- Para forçar NULLs ao final no DESC:
SELECT nome, email FROM CLIENTES
ORDER  BY CASE WHEN email IS NULL THEN 1 ELSE 0 END, email DESC;

-- TOP WITH TIES: inclui empates
SELECT TOP 3 WITH TIES nome_produto, preco FROM PRODUTOS
ORDER  BY preco DESC;  -- se houver empate no 3°, inclui todos

-- ORDER BY posição (evite — pouco legível em manutenção)
SELECT nome_produto, preco FROM PRODUTOS ORDER BY 2 DESC;
-- Melhor: ORDER BY preco DESC (mais claro!)
```

# GROUP BY — Agrupando para Analisar

GROUP BY agrupa linhas com o mesmo valor em uma coluna, permitindo aplicar funções de agregação (COUNT, SUM, AVG...) sobre cada grupo. É a base de qualquer relatório gerencial.

**PEDIDOS (bruto)**

| estado | total |
|---|---|
| SP | 479,80 |
| RJ | 7.199,89 |
| MG | 268,70 |
| SP | 42,50 |
| RS | 89,90 |
| SP | 1.890,00 |

GROUP BY estado →

**Resultado do GROUP BY**

| estado | COUNT(*) | SUM(total) |
|---|---|---|
| MG | 1 | 268,70 |
| RJ | 1 | 7.199,89 |
| RS | 1 | 89,90 |
| SP | 3 | 2.412,30 |

804,10

📌 **Regra do GROUP BY: toda coluna no SELECT que NÃO é função de agregação DEVE aparecer no GROUP BY.**
**Exemplo: SELECT estado, COUNT(*) → GROUP BY estado (obrigatório!)**

# 💻 GROUP BY — Relatórios Reais no SISTEMA_VENDAS

```sql
-- Clientes por estado
SELECT estado, COUNT(*) AS total_clientes
FROM   CLIENTES
GROUP  BY estado
ORDER  BY total_clientes DESC;
-- Produtos por categoria: métricas completas
SELECT id_categoria,
       COUNT(*)           AS qtd_produtos,
       MIN(preco)         AS preco_min,
       MAX(preco)         AS preco_max,
       AVG(preco)         AS preco_medio,
       SUM(estoque)       AS estoque_total,
       SUM(preco*estoque) AS valor_em_estoque
FROM   PRODUTOS
GROUP  BY id_categoria
ORDER  BY valor_em_estoque DESC;
-- Faturamento por mês (GROUP BY com função de data)
SELECT YEAR(data_pedido)  AS ano,
       MONTH(data_pedido) AS mes,
       COUNT(*)           AS total_pedidos,
       SUM(total)         AS faturamento,
       AVG(total)         AS ticket_medio
FROM   PEDIDOS
WHERE  status <> 'CANCELADO'
GROUP  BY YEAR(data_pedido), MONTH(data_pedido)
```

# HAVING — Filtrando Grupos

HAVING filtra GRUPOS após o agrupamento — como o WHERE filtra LINHAS antes. A diferença fundamental: HAVING pode usar funções de agregação (SUM, COUNT...), WHERE não pode.

## WHERE — filtra LINHAS

⏱️ **Quando executa:**

Antes do GROUP BY

✅ **Pode usar:**

Qualquer condição sobre colunas

❌ **Não pode:**

NÃO pode usar COUNT, SUM, AVG...

```
WHERE status <> 'CANCELADO'
```

## HAVING — filtra GRUPOS

⏱️ **Quando executa:**

Depois do GROUP BY

✅ **Pode usar:**

Condições com funções de agregação

❌ **Não pode:**

Não recomendado para colunas simples

```
HAVING COUNT(*) > 2
```

💡 Dica prática: se a condição pode ser aplicada ANTES do agrupamento (sem usar COUNT, SUM...), use WHERE — é mais eficiente. Use HAVING apenas quando precisar filtrar o RESULTADO do agrupamento.

# 💻 HAVING — Filtrando Grupos no SISTEMA_VENDAS

```sql
-- Estados com mais de 1 cliente
SELECT estado, COUNT(*) AS qtd_clientes
FROM   CLIENTES
GROUP  BY estado
HAVING COUNT(*) > 1
ORDER  BY qtd_clientes DESC;
```

```sql
-- Categorias com preço médio acima de R$ 200
SELECT id_categoria,
       COUNT(*)    AS qtd_produtos,
       AVG(preco)  AS preco_medio
FROM   PRODUTOS
GROUP  BY id_categoria
HAVING AVG(preco) > 200.00
ORDER  BY preco_medio DESC;
```

```sql
-- Clientes fiéis: mais de 1 pedido (excluindo cancelados)
SELECT id_cliente,
       COUNT(*)   AS total_pedidos,
       SUM(total) AS gasto_total
FROM   PEDIDOS
WHERE  status <> 'CANCELADO'          -- WHERE filtra antes
GROUP  BY id_cliente
HAVING COUNT(*) > 1                   -- HAVING filtra depois
ORDER  BY gasto_total DESC;
```

```sql
-- Combinando WHERE e HAVING
SELECT id_categoria, COUNT(*) AS qtd, SUM(estoque) AS estoque_total
FROM   PRODUTOS
WHERE  preco > 50                     -- filtra produtos baratos
GROUP  BY id_categoria
HAVING COUNT(*) >= 2                  -- só categorias com 2+ produtos
AND    SUM(estoque) > 10              -- e com estoque suficiente
ORDER  BY qtd DESC;
```

# WHERE vs HAVING — Comparação na Prática

**Mesma pergunta, abordagens diferentes — resultado e performance diferentes:**

## 📌 Clientes de SP com pedidos

### ❌ HAVING para isso (ineficiente)

```
SELECT id_cliente, COUNT(*) AS pedidos
FROM   PEDIDOS
GROUP  BY id_cliente
HAVING id_cliente IN (SELECT id_cliente FROM CLIENTES WHERE
estado='SP')
```

### ✅ WHERE (correto e eficiente)

```
SELECT p.id_cliente, COUNT(*) AS pedidos
FROM   PEDIDOS p
JOIN   CLIENTES c ON p.id_cliente=c.id_cliente
WHERE  c.estado = 'SP'
GROUP  BY p.id_cliente
```

## 📌 Vendedores com faturamento > R$ 3.000

### ❌ WHERE não funciona aqui

```
SELECT id_funcionario, SUM(total)
FROM   PEDIDOS
WHERE  SUM(total) > 3000  -- ERRO!
GROUP  BY id_funcionario
-- Msg 147: aggregate not allowed in WHERE
```

### ✅ HAVING para isso (correto)

```
SELECT id_funcionario, SUM(total) AS fat
FROM   PEDIDOS
WHERE  status <> 'CANCELADO'
GROUP  BY id_funcionario
HAVING SUM(total) > 3000
```

# 💻 Relatório Gerencial Completo — Vendas por Funcionário

```sql
-- Relatório: desempenho de vendedores com todas as métricas
SELECT
    f.nome                              AS vendedor,
    f.cargo,
    COUNT(p.id_pedido)                  AS total_pedidos,
    COUNT(CASE WHEN p.status='ENTREGUE'
               THEN 1 END)              AS pedidos_entregues,
    COUNT(CASE WHEN p.status='CANCELADO'
               THEN 1 END)              AS pedidos_cancelados,
    SUM(CASE WHEN p.status<>'CANCELADO'
             THEN p.total ELSE 0 END)   AS faturamento,
    AVG(CASE WHEN p.status<>'CANCELADO'
             THEN p.total END)          AS ticket_medio,
    MAX(p.total)                        AS maior_pedido,
    ROUND(SUM(CASE WHEN p.status<>'CANCELADO'
         THEN p.total ELSE 0 END)*0.05,2) AS comissao_5pct
FROM   FUNCIONARIOS f
LEFT   JOIN PEDIDOS p ON f.id_funcionario = p.id_funcionario
GROUP  BY f.nome, f.cargo
HAVING SUM(CASE WHEN p.status<>'CANCELADO'
            THEN p.total ELSE 0 END) > 0  -- só quem vendeu algo
ORDER  BY faturamento DESC;
```

# 💻 ROLLUP — Subtotais Automáticos

```sql
-- ROLLUP: gera subtotais automáticos em hierarquia
-- Ex: subtotal por estado E total geral
SELECT estado,
       COUNT(*)           AS qtd_clientes
FROM   CLIENTES
GROUP  BY ROLLUP(estado);
-- Resultado: linhas por estado + 1 linha com NULL = TOTAL GERAL

-- ROLLUP com múltiplos níveis (hierarquia)
SELECT YEAR(data_pedido) AS ano,
       MONTH(data_pedido) AS mes,
       COUNT(*)           AS pedidos,
       SUM(total)         AS faturamento
FROM   PEDIDOS WHERE status <> 'CANCELADO'
GROUP  BY ROLLUP(YEAR(data_pedido), MONTH(data_pedido));
-- Gera: por mês + subtotal por ano + total geral

-- GROUPING(): identifica linhas de subtotal (NULL=subtotal)
SELECT CASE WHEN GROUPING(estado)=1 THEN 'TOTAL GERAL'
            ELSE ISNULL(estado,'Sem estado')
       END AS estado_label,
       COUNT(*) AS qtd
FROM   CLIENTES
GROUP  BY ROLLUP(estado);

-- GROUPING SETS: combinações específicas
SELECT estado, cidade, COUNT(*) AS qtd FROM CLIENTES
GROUP  BY GROUPING SETS((estado,cidade),(estado),());
-- Gera: por estado+cidade, só por estado, e total geral
```

# Window Functions — Classificação sem GROUP BY

Window Functions (funções de janela) calculam valores sobre um conjunto de linhas relacionadas à linha atual — sem colapsar o resultado como GROUP BY. Cada linha permanece no resultado.

`ROW_NUMBER()`

**Numera as linhas sequencialmente dentro do grupo (sem empates)**

```
Sintaxe: ROW_NUMBER() OVER (PARTITION BY estado ORDER BY nome)
```

`RANK()`

**Ranking com lacunas nos empates (1, 2, 2, 4...)**

```
Sintaxe: RANK() OVER (ORDER BY total DESC)
```

`DENSE_RANK()`

**Ranking sem lacunas nos empates (1, 2, 2, 3...)**

```
Sintaxe: DENSE_RANK() OVER (ORDER BY total DESC)
```

💡 **Preview de uso:**

```
SELECT nome_produto, preco, RANK() OVER (ORDER BY preco DESC) AS ranking_preco FROM PRODUTOS;
```

# ✏️ Exercícios Propostos — Aula 09

**01** — Básico

Use GROUP BY para contar clientes por estado. Depois, use HAVING para mostrar apenas estados com mais de 1 cliente. Ordene pelo total de clientes decrescente.

**02** — Básico

Calcule por categoria de produto: quantidade de produtos, preço mínimo, máximo e médio, e total de estoque. Ordene pelo preço médio descendente.

**03** — Médio

Crie um relatório de faturamento por mês: use GROUP BY com YEAR e MONTH, inclua COUNT de pedidos, SUM e AVG do total. Exclua pedidos cancelados com WHERE. Mostre apenas meses com faturamento > R$ 100.

**04** — Médio

Encontre os 'clientes fiéis': mais de 1 pedido não cancelado. Mostre id_cliente, total de pedidos, soma gasta e média por pedido. Ordene pela soma gasta desc.

**05** — Avançado

Use ROLLUP para gerar um relatório de clientes por estado com subtotal por estado e total geral. Use GROUPING() para substituir o NULL da linha de total por 'TOTAL GERAL'. Depois, adicione ROW_NUMBER para numerar cada cliente dentro de seu estado.

# ✅ Respostas Comentadas — Exercícios 03 e 04

```sql
-- EX 03: Faturamento por mês com WHERE e HAVING
SELECT YEAR(data_pedido)  AS ano,
       MONTH(data_pedido) AS mes,
       COUNT(*)           AS total_pedidos,
       SUM(total)         AS faturamento,
       ROUND(AVG(total),2)AS ticket_medio
FROM   PEDIDOS
WHERE  status <> 'CANCELADO'      -- filtra linhas ANTES do GROUP
GROUP  BY YEAR(data_pedido), MONTH(data_pedido)
HAVING SUM(total) > 100           -- filtra grupos DEPOIS do GROUP
ORDER  BY ano, mes;
```

```sql
-- EX 04: Clientes fiéis (mais de 1 pedido)
SELECT p.id_cliente,
       COUNT(*)           AS total_pedidos,
       SUM(p.total)       AS soma_gasta,
       ROUND(AVG(p.total),2) AS media_pedido
FROM   PEDIDOS p
WHERE  p.status <> 'CANCELADO'
GROUP  BY p.id_cliente
HAVING COUNT(*) > 1
ORDER  BY soma_gasta DESC;
```

```sql
-- Para incluir o nome do cliente (JOIN necessário):
SELECT c.nome, COUNT(*) AS pedidos, SUM(p.total) AS gasto
FROM   PEDIDOS p JOIN CLIENTES c ON p.id_cliente=c.id_cliente
WHERE  p.status<>'CANCELADO'
GROUP  BY c.nome HAVING COUNT(*)>1 ORDER BY gasto DESC;
```

# 🏆 Desafio da Aula 09

📋 Contexto: O gerente quer um Dashboard Analítico completo do SISTEMA_VENDAS. Cada query gera um 'card' de métricas para o painel de controle da empresa.

## 🎯 Cards do Dashboard:

**C1** Ranking de produtos: nome, preço, estoque, valor_em_estoque, posição por valor (RANK). Agrupar por categoria com subtotal por categoria (ROLLUP).

**C2** Performance de vendedores: nome, pedidos entregues (COUNT CASE), faturamento (SUM CASE), ticket médio, comissão 5%. Apenas quem tem faturamento > 0 (HAVING).

**C3** Análise de clientes por UF: estado, total clientes, clientes com email (COUNT(email)), % com email (CAST), total gasto pelos clientes daquele estado (subquery). ORDER BY total clientes.

**C4** Evolução mensal: ano, mês (DATENAME), pedidos, faturamento, variação em relação ao mês anterior (LAG window function). Apenas pedidos não cancelados.

**C5** BÔNUS: TOP 3 produtos mais vendidos (por quantidade total em ITENS_PEDIDO) com DENSE_RANK, incluindo nome do produto e categoria.

# 📚 Resumo — Aula 09: Ordenação e Agrupamento

### ⬆️ ORDER BY

ASC(padrão)/DESC. Múltiplas colunas: prioridade da esquerda. Pode usar alias do SELECT

### 📊 GROUP BY

Agrupa linhas iguais. Toda coluna não-agregada no SELECT deve estar no GROUP BY

### 🔍 HAVING

Filtra GRUPOS (pós-agrupamento). Pode usar COUNT, SUM, AVG... WHERE não pode

### 📋 Ordem de execução

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → OFFSET/FETCH

### 🔢 COUNT CASE

COUNT(CASE WHEN cond THEN 1 END) = contagem condicional sem subquery

### 🗂️ ROLLUP

Subtotais automáticos em hierarquia. GROUPING() identifica linhas de total

### 💡 WHERE vs HAVING

WHERE filtra antes (sem agregação). HAVING filtra depois (com agregação)

### 🏆 Window Functions

ROW_NUMBER, RANK, DENSE_RANK: classificam sem colapsar o resultado

# 🚀 Próxima Aula 10: INNER JOIN — Relacionamentos

## Aula 10 — O que vem por aí:

| | |
|---|---|
| 🔗 **O que é JOIN** | Combinando dados de múltiplas tabelas |
| ⚡ **INNER JOIN** | Retorna apenas linhas com correspondência nas duas tabelas |
| 🔀 **JOIN múltiplos** | 3, 4, 5 tabelas em uma única query |
| 📊 **JOIN + GROUP BY** | Relatórios que cruzam dados e agrupam |
| ⚠️ **Produto cartesiano** | O que acontece sem a condição ON — e como evitar |
| 💼 **Casos práticos** | Pedidos com nome de cliente, produto com categoria... |